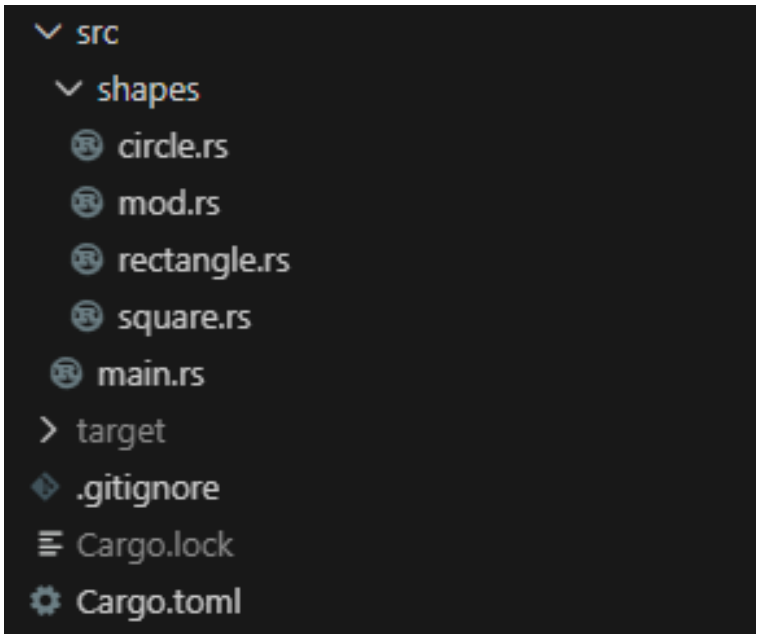
Отчет по лабораторной работе №4 по дисциплине ПиКЯП

Никандров Валентин ИУ5-35Б

Файловая структура проекта:



Листинг кода проекта:

```rust
mod shapes;

use shapes::{Circle, Rectangle, Square};

fn main() {
    let circle = Circle::new(1.0);
    println!("{}", circle);

    let rectangle = Rectangle::new(2.0, 3.0);
    println!("{}", rectangle.to_string());

    let square = Square::new(2.0);
    println!("{}", square);
}
```

```rust
use std::f64::consts::PI;
use crate::shapes::Shape;

pub struct Circle {
    radius: f64,
}

impl Circle {
    pub fn new(radius: f64) -> Self {
        Self { radius }
    }
}

impl Shape for Circle {
    fn calculate_area(&self) -> f64 {
        PI * self.radius * self.radius
    }
}

crate::impl_display_for_shape!(Circle);

#[cfg(test)]
mod tests {
    use std::f64::consts::PI;
    use super::*;

    #[test]
    fn test_circle_area() {
        let circle = Circle::new(3.0);
        let expected_area: f64 = PI * 3.0 * 3.0;
        assert!((circle.calculate_area() - expected_area).abs() < f64::EPSILON)
    }
}
```

```rust
use crate::shapes::Shape;

pub struct Rectangle {
    width: f64,
    height: f64,
}

impl Rectangle {
    pub fn new(width: f64, height: f64) -> Self {
        Self { width, height }
    }
}

impl Shape for Rectangle {
    fn calculate_area(&self) -> f64 {
        self.width * self.height
    }
}

crate::impl_display_for_shape!(Rectangle);

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_rectangle_area() {
        let rectangle = Rectangle::new(2.0, 4.0);
        let expected_area: f64 = 2.0 * 4.0;
        assert_eq!(rectangle.calculate_area(), expected_area)
    }
}
```

```rust
use crate::shapes::Shape;
use crate::shapes::Rectangle;

pub struct Square {
    rectangle: Rectangle,
}

impl Square {
    pub fn new(side: f64) -> Self {
        Self {
            rectangle: Rectangle::new(side, side)
        }
    }
}
```

```rust
impl Shape for Square {
    fn calculate_area(&self) -> f64 {
        self.rectangle.calculate_area()
    }
}

crate::impl_display_for_shape!(Square);

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_rectangle_area() {
        let square = Square::new(2.0);
        let expected_area: f64 = 2.0 * 2.0;
        assert_eq!(square.calculate_area(), expected_area)
    }
}
```

```rust
mod circle;
mod rectangle;
mod square;

pub use circle::Circle;
pub use rectangle::Rectangle;
pub use square::Square;

#[macro_export]
macro_rules! impl_display_for_shape {
    ($type:ident) => {
        impl ::std::fmt::Display for $type {
            fn fmt(&self, formatter: &mut ::std::fmt::Formatter<'_>) ->
::std::fmt::Result {
                write!(
                    formatter,
                    "Shape: {}; Area {};",
                    stringify!($type),
                    self.calculate_area()
                )
            }
        }
    };
}

pub trait Shape {
    fn calculate_area(&self) -> f64;
}
```

Результат работы программы: